

DAY 2 · MODULE 7 · 20 MIN

Day 2 Lab Kickoff

Docs assistant with ticket triage + evaluation

What you'll build

Extend the Day 1 docs assistant into a support triage agent that:

- Answers product questions from documentation (Day 1 baseline)
- Files a support ticket when the docs don't cover it (Module 6)
- Looks up the status of an existing ticket (Module 6)
- Is measured with a retrieval eval and a tool-use eval (Module 3)

All Python. Mock function tools — Day 3 swaps to a real Azure DevOps MCP.

The three parts

Part	Focus	Modules	Time
A	Add a Foundry IQ knowledge source; run retrieval eval	1-3	~40 min
B	Author create_ticket + lookup_status; add tool-use eval	4-6	~50 min
C	Combine knowledge + tools; iterate on instructions	7	~30 min

Total lab time budget: ~2 hours (with breaks).

Part A — Knowledge grounding + retrieval eval

Goal: move the Day 1 assistant from 'prompt-only' to 'grounded in docs.'

- Create a Foundry IQ knowledge source from a small docs corpus (provided)
- Attach it to your Day 1 agent
- Ask three questions the docs answer + two the docs cannot
- Run Retrieval and Groundedness evaluators from Module 3
- Record baseline scores in evals/part_a_baseline.json

Definition of done: Retrieval score ≥ 0.7 on the answerable set; Groundedness ≥ 0.8 .

Part B — Author function tools + tool-use eval

Day 2 · Module 7

Goal: add real actions to the assistant.

- Author `create_ticket(title, body, priority)` — mock backend, Pydantic schema
- Author `lookup_status(ticket_id)` — mock backend, async
- Write pytest tests for both tools in isolation
- Wire the tools to the agent
- Create `evals/tools_golden_set.jsonl` — 6 queries: 2 create, 2 lookup, 2 none
- Run the tool-use eval; iterate on descriptions until you pass

Definition of done: 6/6 tool selections match `tools_golden_set.jsonl`.

Part C — Combine knowledge + tools

Goal: the agent picks the right order.

- Add the Part A knowledge source AND the Part B tools to one agent
- Write instructions using the template in `part_c_combined.py`
- Add three combined queries to `combined_golden_set.jsonl`:
 - Retrieve-then-act (docs → ticket)
 - Act-then-retrieve (lookup → policy explanation)
 - Docs-only (no tool call)
- Iterate on instructions until all three pass

Definition of done: all three combined queries produce the expected trace order.

The starter repo layout

```
labs/day2/
├── README.md           # you're here
├── data/
│   └── docs/           # mock product docs (10 files)
├── python/
│   ├── pyproject.toml  # uv-managed
│   ├── .env.example    # FOUNDRY_PROJECT_ENDPOINT, etc.
│   ├── agent.py        # start here – Day 1 baseline copy
│   ├── tools.py        # your create_ticket + lookup_status
│   └── mock_backend.py  # in-memory ticket store (provided)
├── tests/
│   ├── test_tools.py   # your isolation tests
│   └── test_golden_set.py # your eval runner
└── evals/
    ├── retrieval_eval.py # provided
    ├── tools_golden_set.jsonl # you'll author
    └── combined_golden_set.jsonl # you'll author
```

Prerequisites

Before starting:

- Day 1 lab complete and working
- uv installed (from Day 1)
- FOUNDRY_PROJECT_ENDPOINT in .env (from Day 1)
- Recommended model: gpt-5.6-luna (from Day 1)
- Fresh MSDN subscription with Foundry project — same as Day 1

Setup: `uv sync in labs/day2/python/` then `cp .env.example .env`.

If Day 1 isn't fully working, we'll pair you with a helper before proceeding.

What we WON'T do today

Explicitly out of scope for Day 2 lab:

- Real Azure DevOps integration → Day 3 (MCP)
- Multi-agent orchestration → Day 4
- Production instrumentation / OTel → Day 5
- Approval-mode UX → mentioned in Module 6, not implemented today
- C# implementation → Python only per workshop policy

Keeping scope tight means Parts A-C actually finish in ~2 hours.

Iteration is the point

You will NOT pass every eval on the first try. That's designed in.

Expect to:

- Rewrite a tool description at least once
- Adjust the four-line instructions in Part C at least twice
- See the model pick the wrong tool and fix it via description tightening

The goal isn't to write it right the first time — the goal is to build the eval → iterate → re-eval muscle. That's the day-in day-out workflow.

Support during the lab

- Instructor pod: dedicated Slack channel
- Two TAs on call

Common issues pre-baked into the troubleshooting table:

- Foundry endpoint format
- IQ knowledge source ingestion delay
- Tool selection when descriptions overlap
- .env vs. environment precedence

Ask early. If you're 15 min stuck on something not in the troubleshooting table, flag it.

Takeaways before you start

- Three parts, ~2 hours, Python only
- Definition of done is explicit in every part — chase it, not perfection
- Mock tools now, real MCP tomorrow
- Iteration is the point — build the eval-loop muscle
- Ask early if stuck

Let's build. See you at the debrief.

What's next after the lab

Tomorrow (Day 3): we swap mocks for real integrations.

- `create_ticket` becomes a real Azure DevOps MCP call
- `lookup_status` becomes a real Azure DevOps MCP query
- Same conceptual pattern you built today — different backend
- Adds MCP tool authoring on top of Day 2's function tool authoring

Everything you build today carries forward.